

Python Bridge – Addon-Dokumentation

Version 0.2.1 | Godot 4.2+ (getestet 4.7.x) + Python 3.8+ | WebSocket-basierte Python-Prozess-Instanzen
mit Task-Manager

Inhalt

1. Übersicht
2. Architektur
3. Installation in einem neuen Projekt
4. GDScript-API-Referenz
5. Task Manager & Scheduler
6. Datentransport & Typen
7. Python-Seite
8. Editor-Integration
9. Lebenszyklus, Provisioning & Fehlerbehandlung
10. Troubleshooting

1. Übersicht

Die Python Bridge startet aus Godot heraus einen oder mehrere Python-Subprozesse (je Instanz ein eigener Prozess) und kommuniziert mit ihnen über WebSocket auf `127.0.0.1`. Sie übernimmt automatisch:

- **venv-Verwaltung:** legt pro Projekt ein virtuelles Environment unter `python_bridge/venv/` an (einmalig).
- **Dependency-Management:** installiert `websockets` + frei konfigurierbare Pakete (z. B. `numpy`) per `pip`.
- **Python-Erkennung:** findet eine installierte Python (3.8+) selbst, ohne fest verdrahteten Pfad.
- **Task Manager:** zentrale Warteschlange mit Prioritäten, Timeout, Cancel, Retry und Backpressure.
- **Frame-Synchronisation:** Python-Antworten werden gepuffert und kontrolliert im Haupt-Thread verarbeitet – kein Frame-Stall.
- **Task-Batching:** kurz aufeinanderfolgende Aufrufe werden optional zu einem Paket zusammengefasst.
- **Health Monitoring & Crash-Restart:** Ping/Pong-Überwachung, automatischer Neustart mit Exponential Backoff.
- **Typisierten Transport:** JSON für Skalare, Binär-Chunks für große Daten (NumPy-Arrays, Bytes, Bilder).
- **Persistente Skript-Kontexte:** Skripte und deren Zustand (Importe, Variablen, Funktionen) bleiben in der Instanz erhalten.
- **Hot Reload:** geänderter Python-Code wird erkannt und per Konfiguration neu geladen.
- **Editor-Integration:** Python-Editor-Dock mit Syntax-Highlighting und deterministischer GDScript-Wrapper-Generierung.

2. Architektur



```

└─ BridgeHealthMonitor                                Text = JSON · Binary =
└─ server (asyncio + Worker-Thread)

└─ BridgeProvisioner (venv, pip)                      Header + Chunks
└─ executor (Kontexte, Source-Hash)

└─ introspection (AST)

└─ serializer · protocol (v2)

```

Der Ablauf: Godot startet den Python-Server als Subprozess. Der Server schreibt Port + PID in `python_bridge/tmp/<instanz>.json`. Godot liest die Datei, verbindet sich per WebSocket, sendet ein `hello` und gilt nach der `hello_ack`-Antwort als `READY`.

Invariante: Python wartet nie auf einen Godot-Frame, und Godot wartet nie auf Python. Eingehende Ergebnisse werden im Scheduler gepuffert und innerhalb eines definierten Sync-Punkts (`max_max_results_per_frame` pro Frame) verarbeitet. Jede Anfrage trägt eine eindeutige `id`; Antworten werden darüber den Tasks zugeordnet.

Nachrichten-Typen (Protokoll v2): `hello` → `hello_ack · task` / `batch` → `task_result` / `task_error` / `batch_result · cancel` → `cancel_ack · ping` → `pong · reload` → `reload_ack · introspect` → `introspect_result · shutdown` → `shutdown_ack · event · status`

3. Installation in einem neuen Projekt (Schritt für Schritt)

Die folgenden fünf Schritte führen ohne Vorkenntnisse durch die komplette Einrichtung. Am Ende dieses Kapitels kannst du die erste Python-Funktion aus einem GDScript-Skript heraus aufrufen.

3.1 Voraussetzungen

- Godot 4.2+ (getestet mit 4.7.x) – die verwendeten APIs existieren ab 4.2.
- Python 3.8+ muss installiert sein. Die Bridge findet sie automatisch über `PATH`, `PYTHON_PATH` oder `python_executable` – es wird kein fester Pfad benötigt.

3.2 Schritt 1: Addon in das Projekt kopieren

Den kompletten Ordner `addons/python_bridge/` in das Wurzelverzeichnis des Zielprojekts kopieren, sodass er unter `res://addons/python_bridge/` liegt (im Godot-**FileSystem**-Dock sichtbar).

3.3 Schritt 2: Das Plugin aktivieren

1. Godot öffnen (das Projekt muss neu gescannt werden, damit das Addon im **FileSystem**-Dock unter addons/ auftaucht).
2. Hauptmenü oben: **Projekt** → **Projekteinstellungen...**
3. Im Dialog oben auf den Reiter **Plugins** klicken.
4. Genau ein Eintrag: **Python Bridge** → rechts das Häkchen bei **Aktivieren** setzen.
5. Godot fragt ggf. nach einem **Editor-Neustart** → zustimmen.

Wichtig: Ohne diesen Schritt existiert das Dock noch nicht. Die Aktivierung bewirkt zwei Dinge: (1) Der

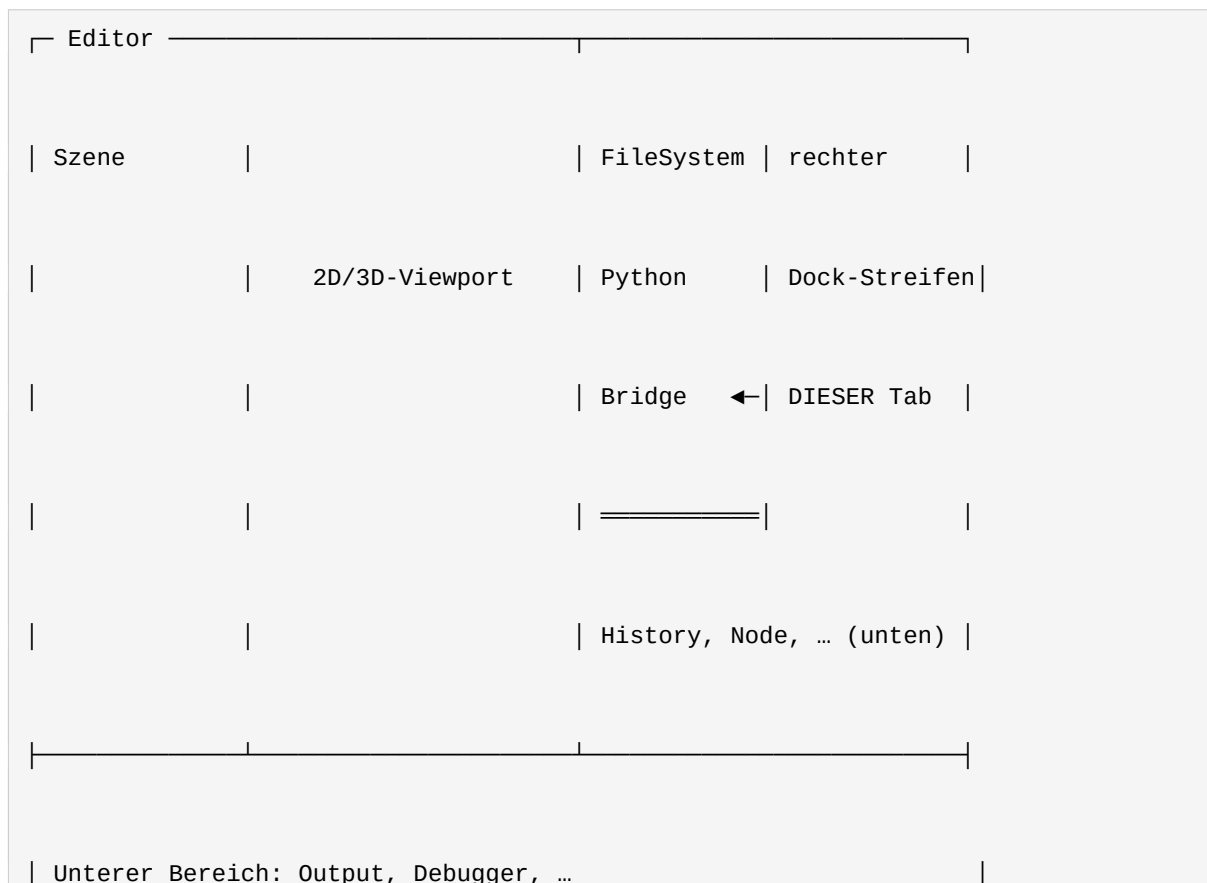
Autoload-Singleton PythonBridge wird registriert (Projekt → Projekteinstellungen → Autoload) – von

dort rufen alle deine Skripte `PythonBridge.call_script(...)` usw. auf. (2) Das Editor-Dock wird in den

rechten Dock-Streifen gehängt.

3.4 Schritt 3: Das „Python Bridge“-Dock finden

Das Dock ist ein Panel in der **rechten Dock-Leiste des Editors** – derselbe vertikale Streifen, in dem auch **FileSystem** (Standard: oberer Bereich) und **History / Node / Editor-Doku** (unterer Bereich) liegen. Genauer Ort: **rechts, oberer Slot, direkt neben dem FileSystem-Tab**.



So erkennst du es:

- Am **rechten Fensterrand** steht eine **vertikale Spalte von Tab-Beschriftungen** (gedrehter Text; solange das Panel zugeklappt ist, ist nur die Beschriftung sichtbar).
- Klicke auf den vertikalen Tab „**Python Bridge**“ → das Panel fährt aus und enthält von oben nach unten:
 - a. **Kopfzeile:** Refresh + New script + Statusanzeige
 - b. **Skriptliste** (alle .py-Dateien aus `res://python_bridge/scripts/`)
 - c. **Code-Editor** mit Python-Syntax-Highlighting
 - d. **Aktionszeile:** Save · Run · Generate wrapper · Hot reload
 - e. **Log-Bereich**

Falls der Tab fehlt oder verschoben wurde: **Editor** → **Editor-Layout** → **Standard-Layout** setzt alle Docks zurück.

3.5 Schritt 4: Erste Python-Datei über das Dock anlegen

1. Im Dock auf **New script** klicken → Name eingeben (z. B. hallo) → es entsteht `res://python_bridge/scripts/hallo.py` – eine **ganz normale .py-Datei** (auch außerhalb von Godot ausführbar).
2. Code eintippen bzw. einfügen → **Save**.

Button im Dock

Wirkung

Refresh	Skriptliste neu einlesen (erkennt auch von Hand angelegte Dateien).
New script	Neue .py-Datei im Workspace anlegen.
Save	Aktuell geöffneten Code speichern.
Run	Datei sofort ausführen (<code>execute_script</code>), Ausgabe erscheint im Log.
Generate wrapper	GDScript-Klasse für das Skript erzeugen (siehe Kapitel 8; optional, erspart <code>call_script</code> -Boilerplate).
Hot reload	Geänderten Code ohne Neustart neu laden (je nach <code>hot_reload_mode</code>).

3.6 Schritt 5: Was beim ersten Start automatisch passiert

Beim ersten `await PythonBridge.start_instance("default")` (aus `GDScript`, siehe Kapitel 4) legt die Bridge automatisch an bzw. startet:

- `res://python_bridge/` mit `scripts/`, `tmp/`, `config/`, `venv/`,
- eine Python-venv (falls nicht vorhanden),
- `websockets>=11` + konfigurierte Dependencies (nicht-blockierend, Log in `tmp/pip.log`),
- den Python-Server als Subprozess + WebSocket-Verbindung bis `READY`.

Hinweis: Du startest **niemals selbst ein Terminal** für Python – die Bridge verwaltet Prozess, Verbindung

und Tasks vollständig. Das venv liegt im Projektordner; damit Godot es nicht als Ressourcen importiert,

enthält `python_bridge/venv/` eine `.gdignore`-Datei. Das Plugin kopiert die Python-Quellen bei jedem

Start nach `python_bridge/bridge/`, damit Quell- und Laufzeit-Version immer identisch sind.

4. GDScript-API-Referenz

Alle Funktionen sind Instanz-Methoden des Autoload-Singletons `PythonBridge` und dürfen daher `await` verwenden. Aufrufe, die Python ausführen, liefern ein `PythonBridgeResult` (siehe 4.3).

4.1 PythonBridge (Autoload) – Konfiguration & Instanzen

Funktion	Beschreibung
<code>configure(cfg: Dictionary)</code>	Globale Einstellungen setzen (siehe Tabelle in Kapitel 5.1). Alle Schlüssel sind optional; Werte werden typisiert normalisiert.
<code>config() → Dictionary</code>	Aktive Konfiguration zurückgeben (Kopie).
<code>workspace_dir() → String</code>	Aktueller Workspace-Pfad (<code>res://python_bridge</code> als Standard).
<code>start_instance(name := "default") → Result</code>	Startet (bzw. holt) eine Instanz und wartet bis <code>READY</code> (Timeout 180 s). Liefert <code>PythonBridgeResult</code> .
<code>get_instance(name := "default")</code>	Liefert die <code>BridgeInstance</code> (oder <code>null</code>), ohne sie zu starten.

<code>instance_status(name := "default") → String</code>	Status-String: none provisioning starting connecting handshake ready crashed restarting stopping stopped error.
<code>stop_instance(name := "default") · stop_all()</code>	Graceful Shutdown einer/aller Instanzen (sendet shutdown, wartet auf Ack, killt bei Timeout).
<code>shutdown() · shutdown_now()</code>	Blockiert neue Tasks und stoppt alle Instanzen; <code>shutdown_now()</code> erzwingt den sofortigen Stop (keine Zombies).
<code>poll()</code>	Treibt alle Instanzen und den Scheduler pro Frame (automatisch in <code>_process</code> und vom Editor-Plugin).
<code>static system() → Node</code>	Statischer Helfer: findet den Autoload-Node über den Szenenbaum.

Signale: `pulse` (jeden Frame) · `instance_state_changed(instance, state)` · `task_done(task)` (jeder terminale Task) · `bridge_event(instance, event)` (Python → Godot-Events).

4.2 PythonBridge – Skript- & Task-API

Funktion	Beschreibung
<code>execute(code, input := {}, instance := "default", timeout_sec := 30.0)</code>	Führt temporären Python-Code in einem Wegwerf-Kontext aus (Konvention: <code>input / result</code>).
<code>create_script(script_id, code, subfolder := "")</code>	Schreibt ein dauerhaftes Skript nach <code>res://python_bridge/scripts/<subfolder>/<id>.py</code> (überschreibt). Liefert <code>PythonBridgeResult</code> .
<code>get_script_source(script_id) → String</code>	Quelltext des Skripts (leer, wenn nicht vorhanden).
<code>execute_script(script_id, input := {}, instance := "default", timeout_sec := 30.0)</code>	Führt ein dauerhaftes Skript aus (Konvention: <code>input / result</code>).
<code>call_script(script_id, function, args := [], kwargs := {}, instance :=</code>	Ruft eine benannte Funktion im Skript-Kontext auf; der Rückgabewert ist das Ergebnis. Der Kontext wird per Source-Hash nur bei Änderung

<code>"default", timeout_sec := 30.0)</code>	neu definiert.
<code>define_script(script_id, instance := "default", timeout_sec := 30.0)</code>	Lädt ein Skript vorab in den Kontext (ohne Ausführung).
<code>script_path_for(script_id, subfolder) · resolve_script_path(script_id)</code>	Pfad-Helfer; erlaubt volle Pfade oder <code>id / unterordner/id</code> .
<code>submit_task(task: PythonBridgeTask) → Result</code>	Reicht einen Task in den Task Manager ein (Backpressure-Check; Ergebnis kommt über <code>task.done</code>).
<code>cancel_task(task_id) → bool</code>	Bricht einen Task ab (Queue- oder In-Flight).
<code>get_task(task_id) → PythonBridgeTask</code>	Task-Objekt nach ID (für Status/Ergebnis).
<code>hot_reload_script(script_id) → Result</code>	Löst Hot Reload aus – je nach <code>hot_reload_mode</code> : <code>none</code> (deaktiviert), <code>reload_context</code> (Kontext wird invalidiert; Standard) oder <code>restart_instance</code> (Instanz neu starten).
<code>introspect_script(script_id, instance := "default") → Result</code>	Fragt die Funktions-Signaturen per AST ab (ohne das Skript auszuführen); Basis für die Wrapper-Generierung.

4.3 PythonBridgeTask

Ein Task durchläuft die Zustände `QUEUED → RUNNING → COMPLETED | FAILED | CANCELLED | TIMEOUT`. Ergebnis und Fehler sind über das Signal `done(result: PythonBridgeResult)` abrufbar (genau einmal pro Task).

Feld	Beschreibung
<code>id · instance_id</code>	Eindeutige Task-ID; <code>instance_id = ""</code> bedeutet Auto-Zuordnung an eine beliebige bereite Instanz.
<code>priority</code>	Ganzzahl, 0 = höchste Priorität (stabile Reihenfolge bei gleicher Priorität).
<code>command</code>	<code>run call define</code>

<code>context_id · source · function · args · kwargs · input</code>	Ausführungsdaten des Tasks.
<code>timeout_ms</code>	Maximale Gesamtlebensdauer (Queue + Ausführung); Überschreitung → TIMEOUT.
<code>max_retries · retries_left · retry_policy</code>	Retry-Konfiguration (Policy: none connection_error process_error all).
<code>batchable</code>	Erlaubt dem Scheduler, den Task in ein Batch-Frame zu fassen (Standard true).
<code>result · meta · state</code>	Endergebnis, Metadaten (z. B. duration_ms) und aktueller Zustand.

Konstrukturen (statisch): `PythonBridgeTask.make_call(id, context, source, function, args, kwargs, timeout_ms, priority := 0), make_run(...), make_define(...)`.

4.4 BridgeInstance

Eine Instanz = ein Python-Subprozess + ein WebSocket-Kanal. Meist reicht die PythonBridge-Facade; für Spezialfälle kann man die Instanz direkt nutzen.

Funktion	Beschreibung
<code>send_message(msg: Dictionary) → Error</code>	Sendet eine Protokoll-Nachricht (nur im READY-Zustand).
<code>wait_ready(timeout := 180) → bool</code>	Wartet (via await auf <code>process_frame</code>) bis READY.
<code>status_text() · is_active() · is_ready() · is_ready_immediately()</code>	Zustandsprüfungen (siehe 4.1 für Status-Strings).
<code>last_error_message() → String</code>	Letzter Fehlertext (Provisioning-/Verbindungsfehler).
<code>stop() · shutdown_now()</code>	Graceful bzw. sofortiger Stop (inkl. Prozess-Kill bei Bedarf).

Signale: `instance_ready · state_changed(instance, state) · message_received(instance,`

parsed) · lost(instance, error). Zustandsmaschine: NONE → PROVISIONING → STARTING → CONNECTING → HANDSHAKE → READY (bei Fehler/Crash: CRASHED → RESTARTING → STARTING mit Backoff, oder ERROR; bei Stop: STOPPING → STOPPED).

4.5 PythonBridgeResult

Feld	Beschreibung
<code>ok / is_ok() / is_error()</code>	Erfolg? Fehler?
<code>status</code>	"ok", "error" (Python-Exception), "timeout", "not_ready", "down" (Prozess getrennt), "internal" (Tool-Problem), "cancelled".
<code>value</code>	Ergebnis bei Erfolg (typisiert, siehe Kapitel 6).
<code>error</code>	Dictionary {code, type, message, traceback, task_id?, instance_id?} bei Fehlern (Taxonomie in 9.2).
<code>error_message() · error_code()</code>	Kurztext bzw. Fehlercode des Fehlers.
<code>request_id · task_id · instance_id · meta</code>	Zuordnungs- und Meta-Daten (z. B. duration_ms).

5. Task Manager & Scheduler

5.1 Konfiguration (PythonBridgeConfig)

Schlüssel	Standard	Bedeutung
<code>autostart</code>	<code>false</code>	Instanz(en) beim Plugin-Start automatisch hochziehen.
<code>workspace_dir</code>	<code>res://python_bridge</code>	Workspace (scripts/, tmp/, config/, venv/, bridge/).
<code>python_executable</code>	<code>""</code>	Leer = automatische Python-Erkennung.
<code>dependencies</code>	<code>[]</code>	Zusätzliche pip-Pakete (z. B.

		["numpy"]).
max_queued_tasks	1000	Backpressure: maximale Queue-Länge.
max_inflight_per_instance	1	Maximale gleichzeitig laufende Einheiten pro Instanz.
max_payload_bytes	64 MiB	Maximale Task-Payload (Quelle + Input/Args).
task_timeout_ms	30000	Standard-Timeout für Tasks.
max_retries · retry_policy · retry_delay_ms	0 · connection_error · 250	Retry-Konfiguration des Task Managers.
max_batch_size · max_batch_delay_ms	32 · 32	Batch-Fenster: Größe und maximale Wartezeit.
max_dispatch_per_frame · max_results_per_frame · max_inbox_size	16 · 64 · 512	Frame-Budgets des Schedulers; volle Inbox throtzelt den Dispatch.
connect_timeout_ms · provision_venv_timeout_ms · provision_pip_timeout_ms · shutdown_timeout_ms	20000 · 120000 · 300000 · 3000	Zeitlimits für Verbindung, Provisioning und Shutdown.
health_check_interval_ms · health_missed_pong_limit	5000 · 3	Health Monitoring: Ping-Kadenz und verpasste Pongs bis zum Crash.
max_restart_attempts · restart_base_delay_ms · restart_backoff_factor · stable_uptime_ms	3 · 500 · 2 · 30000	Crash-Restart: Versuche, Backoff (500 → 1000 → 2000 ms), Reset nach stabiler Laufzeit.
hot_reload_mode	"reload_context"	none reload_context restart_instance.
auto_generate_wrappers · wrapper_dir	false · res://python	Automatische Wrapper-Generierung (Standard:

5.2 Verhalten

- **Priorität:** 0 = höchste; stabile Reihenfolge (FIFO) bei gleicher Priorität.
- **Timeout:** ein Timer pro Task (Queue + Ausführung); bei Überschreitung wird ein CANCEL best-effort gesendet und der Task auf TIMEOUT gesetzt.
- **Cancel:** Queue-Tasks werden sofort entfernt; laufende Tasks werden markiert und als CANCELLED abgeschlossen (späte Ergebnisse werden verworfen).
- **Retry:** nur gemäß `retry_policy`; `retries_left` wird dekrementiert, der Task mit `retry_delay_ms` neu eingereiht.
- **Backpressure:** `max_queued_tasks` und `max_payload_bytes` lehnen Tasks mit strukturiertem Fehler ab (Queue voll / Payload zu groß).
- **Frame-Sync:** Ergebnisse landen in einer begrenzten Inbox; pro Frame werden max. `max_results_per_frame` verarbeitet. Ein wachsender Backlog throttle den Dispatch – stabile Framezeiten statt maximaler Antwortgeschwindigkeit.
- **Batching:** kompatible, batchbare Tasks derselben Priorität an dieselbe Instanz werden bis `max_batch_size` oder `max_batch_delay_ms` gesammelt. Ein einzelner Task wartet nie auf weitere; eine höhere Priorität bricht das Fenster ab. Reihenfolge und Semantik bleiben erhalten; ein Fehler innerhalb eines Batches betrifft nur den betroffenen Item (separate Ergebnisse).
- **Crash-Behandlung:** bei Verbindungsabbruch oder Prozess-Ende werden In-Flight-Tasks gemäß Policy gefailt oder erneut eingereiht; die Instanz startet mit Exponential Backoff neu (Counter-Reset nach `stable_uptime_ms`).

6. Datentransport & Typen

Skalare bleiben lesbares JSON. Strukturierte Werte werden als getaggte Objekte `{"$pb": "<tag>", ...}` kodiert. Große Binär-Blobs (> 512 Bytes) wandern nicht als Base64 in den JSON-Header, sondern als Binär-Chunks im Frame (`U32LE(Header-Länge) + HeaderJSON + [U32LE(Chunk-Länge) + Chunk]*`). Die Typ-Mapping-Tabelle ist zentral in `PythonBridgeTypeMapper` und um benutzerdefinierte Typen erweiterbar (`register(tag, encode_fn, decode_fn, custom_class := "")`).

Python → Godot

None / bool / int
/ float / str

list / tuple /
set

Godot-Typ

null / bool / int / float / String

Array

<code>dict</code>	Dictionary (Schlüssel müssen str/int/float/bool sein)
<code>bytes</code> / <code>bytearray</code>	<code>PackedByteArray</code>
<code>numpy.ndarray</code> (diverse dtypes)	<code>PackedFloat64Array</code> / <code>PackedFloat32Array</code> / <code>PackedInt32Array</code> / <code>PackedInt64Array</code> / <code>PackedByteArray</code> (je nach dtype)
<code>numpy.integer</code> / <code>numpy.floating</code>	<code>int</code> / <code>float</code>
beliebiges anderes Objekt	<code>String</code> (repr-Fallback, Tag <code>pyobject</code>)

Godot → Python

Python-Typ

<code>Vector2/3/4</code> · <code>Color</code> · <code>Transform3D</code>	<code>list</code> der Komponenten (z. B. <code>[x, y, z]</code>)
<code>PackedByteArray</code> (groß)	<code>bytes</code> (als Chunk)
<code>PackedInt/Float-Arrays</code>	<code>bytes</code> (roh, für <code>numpy.frombuffer</code>)
<code>Array</code> / <code>Dictionary</code>	<code>list</code> / <code>dict</code>
<code>Image</code>	rohe Pixel-Daten + Metadaten (w, h, format)

Endianness: Längen-Präfixe im Binärformat sind Little-Endian auf beiden Seiten (`struct.pack("<I", ...)` in Python, `decode_u32` / explizite Byte-Kodierung in Godot).

7. Python-Seite

7.1 Skript-Konventionen

- `execute` / `execute_script`: Das Skript erhält die Eingabe in der Variablen `input` und legt das Ergebnis in `result` ab.
- `call_script`: Die benannte Funktion wird mit `args` / `kwargs` aufgerufen; ihr Rückgabewert geht an Godot.
- **Persistente Kontexte:** Jede `context_id` hat einen eigenen Namespace. Der Executor speichert einen Source-Hash; `call_script` re-definiert den Kontext nur, wenn sich der Quelltext geändert hat.

- **Fehler:** Python-Exceptions werden strukturiert gemeldet (`{code, type, message, traceback}`) – der Server-Prozess bleibt am Leben.
- **stdout/stderr:** werden pro Task erfasst und im Ergebnis mitgeliefert.

7.2 Aufbau des Python-Pakets

Datei	Zweck
<code>run_server.py</code>	Entry-Point: CLI-Parameter (<code>--bind --port --tmpdir --tag</code>), startet den asyncio-Server.
<code>python_bridge/server.py</code>	WebSocket-Server: routet <code>task / batch / cancel / reload / introspect / ping / hello / shutdown</code> ; Nutzer-Code läuft in genau einem Worker-Thread pro Instanz, damit der Event-Loop reaktionsfähig bleibt; Watchdog beendet den Prozess nach Shutdown/Verbindungsabbruch.
<code>python_bridge/executor.py</code>	Verwaltet Kontexte, führt <code>define / run / call</code> aus (Source-Hash), fängt Exceptions, capturet stdout/stderr, unterstützt Cancellation.
<code>python_bridge/introspection.py</code>	AST-basierte Funktions-Signatur-Analyse (führt nie Code aus) – Grundlage der Wrapper-Generierung.
<code>python_bridge/protocol.py</code>	v2-Nachrichtentypen und Batch-Item-Dekodierung.
<code>python_bridge/serializer.py</code>	<code>encode_obj / decode_obj</code> : typisierte Serialisierung inkl. NumPy-Arrays und Chunks.
<code>requirements.txt</code>	Minimale Abhängigkeit: <code>websockets>=11</code> (weitere Pakete pro Projekt konfigurierbar).

8. Editor-Integration

Das Plugin registriert den Autoload `PythonBridge` und hängt ein leichtgewichtiges Dock-Panel (rechts oben) an. Der Editor spricht ausschließlich mit der Facade – keinerlei Runtime-Logik im Editor.

Element	Beschreibung
Dock-Panel	Skriptliste (auto-refresh), CodeEdit mit Python-Syntax-

(editor/python_editor.gd)

Highlighting, Aktionen: **Save** · **Run** · **Generate wrapper** · **Hot reload** · **Refresh** · **New script**, Status + Log, mtime-Watcher für externe Änderungen.

Syntax-Highlighting
(editor/python_syntax_highlighter.gd)

Keywords, Builtins, Strings, Kommentare, Zahlen, Dekoratoren.

Wrapper-Generator
(editor/wrapper_generator.gd)

Deterministisch (gleiches Schema → identische Bytes);
Marker-Header; überschreibt nur markierte Dateien;
class_name-Kollisions-Check; Python-Defaults bleiben
autoritativ (Sentinel-Check).

Wrapper-Beispiel: Aus `def calculate(a: int, b: int) -> int: return a + b` erzeugt der Generator ein GDScript wie `bridge.calculate(2, 3)`. Der originale `.py`-Code bleibt unverändert und außerhalb von Godot lauffähig. Wrapper werden nur erzeugt, wenn man sie anfordert (Button bzw. `auto_generate_wrappers`).

9. Demo-Szene & Beispiele

Das Repository enthält eine fertige Demo-Szene (`example/demo_scene.tscn`) mit Nodes und angehängten Skripten – öffnen und **F6** (Aktuelle Szene ausführen) drücken. Der Button **Bridge starten** startet die Python-Runtime, danach kann man die einzelnen Aspekte per Button ausprobieren. Jedes Demo-Node ist ein eigenes Skript unter `example/demo/`:

Node / Skript

Demonstriert

DemoRoot –
`demo_root.gd`

Orchestriert Bridge-Start, Workspace-Sync (`example/scripts/*.py` → `python_bridge/scripts/`), Shutdown.

DemoBasic –
`demo_basic.gd`

`call_script` (Args/Kwargs), `execute_script` (input/result),
`define_script` – inkl. strukturierter Typen (Array, Dictionary).

DemoTasks –
`demo_tasks.gd`

Low-Level-Task-API: `PythonBridgeTask.make_call`, `submit_task`,
done-Signal, Priorität, Timeout, `cancel_task`.

DemoBatch –
`demo_batch.gd`

8 schnelle Aufrufe → Task-Batching
(`max_batch_size/max_batch_delay_ms`) mit erhaltener Reihenfolge.

DemoErrors –
`demo_errors.gd`

Strukturierte Fehler: Python-Exception
(Typ/Message/Traceback/Task-ID), Timeout, unbekanntes Skript.

DemoMulti –
demo_multi.gd

Zweite Python-Instanz (worker) parallel zur Standard-Instanz, Task-
Verteilung, Statusabfrage, Stop.

Die Beispiel-Python-Skripte liegen in `example/scripts/`: `demo_skript.py` (calculate, greet, fibonacci, vector_list, boom, slow, big_blob) und `crash_skript.py` (`crash()` beendet den Python-Prozess absichtlich – zeigt Crash-Erkennung und Exponential-Backoff-Restart der Bridge).

10. Lebenszyklus, Provisioning & Fehlerbehandlung

9.1 Lebenszyklus & Provisioning

1. **PROVISIONING** – Workspace-Ordner anlegen (`scripts/`, `tmp/`, `config/`), Python erkennen (≥ 3.8), Bridge-Python nach `python_bridge/bridge/` kopieren.
2. **venv** – falls nicht vorhanden: `python -m venv .../venv` (nicht-blockierend, Timeout 120 s).
3. **pip** – `pip install websockets>=11` + konfigurierte Dependencies (Timeout 300 s, Log in `tmp/pip.log`), anschließend Import-Verifikation.
4. **STARTING** – `run_server.py` als Subprozess starten; Server schreibt `tmp/<instanz>.json` (Port + PID).
5. **CONNECTING** → **HANDSHAKE** – WebSocket-Verbindung aufbauen, `hello` senden.
6. **READY** – `hello_ack` empfangen; Tasks sind jetzt möglich.

Crash & Restart: Ein unerwarteter WebSocket-Close oder ein toter Subprozess im READY-Zustand löst die Restart-Policy aus: `max_restart_attempts` Versuche mit Exponential Backoff (500 ms → 1000 ms → 2000 ms, konfigurierbar), Counter-Reset nach `stable_uptime_ms`. Es bleiben keine Zombie-Prozesse zurück (vor jedem Neustart wird ein eventuell noch laufender Prozess beendet).

Graceful Shutdown: Beim Beenden von Godot (`shutdown()/_exit_tree`) werden neue Tasks blockiert, `shutdown` an alle Instanzen gesendet, auf `shutdown_ack` (bzw. Timeout `shutdown_timeout_ms`) gewartet und Prozesse notfalls zwangsweise beendet.

9.2 Fehler-Taxonomie

Jeder Fehler trägt einen Code aus der Taxonomie (`PythonBridgeErrorHandler`). Der Status eines `PythonBridgeResult` wird daraus abgeleitet.

Code	Bedeutung	Status
<code>PYTHON_EXCEPTION · TASK_ERROR</code>	Python-Exception bzw. Task-Fehler	error
<code>TIMEOUT_ERROR</code>	Antwort kam nicht	timeout

	rechtzeitig	t
PROCESS_ERROR · CONNECTION_ERROR	Prozess-/ Verbindungsfehler	dow n
DEPENDENCY_ERROR · SERIALIZATION_ERROR · PROTOCOL_ERROR · BRIDGE_ERROR	Infrastrukturfehler	int ern al

Eine Python-Exception enthält mindestens: Exception-Typ, Message, Traceback, Task-ID, Instanz-ID – nie nur „Something went wrong“.

11. Troubleshooting

Problem / Status	Ursache & Lösung
status = "not_ready"	Instanz wurde nicht gestartet oder noch nicht READY – vor Aufrufen <code>await start_instance()</code> nutzen bzw. <code>wait_ready()</code> .
status = "timeout"	Antwort kam nicht in <code>timeout_sec</code> (Standard 30 s). Timeout erhöhen oder Python-Code beschleunigen.
status = "error"	Python-Exception im Nutzer-Code – Details in <code>result.error</code> (type / message / traceback).
status = "down"	Python-Prozess getrennt (WebSocket geschlossen) – z. B. Absturz im Server oder <code>stop()</code> . Prozess-Log prüfen; die Instanz restartet gemäß Backoff-Konfiguration.
status = "cancelled"	Task wurde vor Abschluss abgebrochen (<code>cancel_task</code> oder Shutdown).
„Kein Python gefunden“	<code>python_executable</code> setzen, <code>PYTHON_PATH</code> definieren oder Python zum PATH hinzufügen.
venv/pip hängt	Timeout 120/300 s; Details in <code>python_bridge/tmp/pip.log</code> . Ggf. venv-Ordner löschen und neu provisionieren lassen.
„Python-Prozess getrennt“ beim ersten Aufruf	Meist ein Server-/Protokollproblem beim Binär-Versand – Version von <code>addons/python_bridge/python</code> (Quelle) und <code>python_bridge/bridge</code> (Laufzeit-Kopie) muss identisch sein; das Plugin kopiert bei jedem Start automatisch.

Werte kommen als
String an

Objekt war nicht serialisierbar → `pyobject`-Fallback (`repr`). Nur
JSON-fähige Typen, `bytes`, `ndarray`, Listen/Dicts verwenden.

Compiler-Fehler
„Could not resolve
class ... parser
error“

Alter `.godot`-Cache aus einer früheren Addon-Version. Projekt
einmal schließen, `.godot/` löschen und erneut öffnen (die `.uid-`
Dateien sind im Repo enthalten).

Python Bridge v0.2.1 – dokumentiert nach dem Quellcode in `addons/python_bridge/` (Godot-Seite: `core/`,
`editor/`; Python-Seite: `python/`). Weitere Details: `docs/PRAXIS.md` (Schnellstart in 10 Schritten),
`docs/ARCHITEKTUR.md`, `docs/API.md`, `docs/INSTALLATION.md`, `docs/TROUBLESHOOTING.md`,
`docs/CHANGELOG.md`.